BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE, PILANI
DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION SYSTEMS

Compiler Construction (CS F363)
II Semester 2025-26
Compiler Project
Coding Details
(March 02, 2026)

Group Number
1

1. Team Members Names and IDs
   ID 2023A7PS0021P Name Romit Jain
   ID 2023A7PS0025P Name Soham Vinay Deshmukh
   ID 2023A7PS0030P Name Nishant Amarish Pradhan
   ID 2023A7PS0242P Name Devesh Saraogi
2. Mention the names of the Submitted files :

1 grammar.txt	7 parser.h	13 testcase3.txt
2 codingDetails.pdf	8 parser.c	14 testcase4.txt
3 lexer.c	9 driver.c	15 testcase5.txt
4 lexer.h	10 makefile	16 testcase6.txt
5 lexerDef.h	11 testcase1.txt	17 testcase7.txt
6 parserDef.h	12 testcase2.txt	18 testcase8.txt

3. Total number of submitted files (including copy the pdf file of this coding details pro forma) : 18
   (All files should be in ONE folder named as Group_#)

4. Have you compressed the folder as specified in the submission guidelines? (yes/no) Yes

5. **Lexer Details:**
   [A]. Technique used for pattern matching: DFA generated from regular expressions, longest-matching principle.
   [B]. Keyword Handling Technique: Reserved-word table (hash table) — identifiers are looked up after lexeme recognition.
   [C]. Hash function description, if used for keyword handling: Follows the djb2 hashing algorithm, start with 5381, for each char do hash = ((hash << 5) + hash) + c, then index = hash % table size.
   [D]. Have you used twin buffer? (yes/ no) yes
   [E]. Error handling and reporting (yes/No): yes
   [F]. Describe the errors handled by you: invalid/unknown tokens, unterminated string or comment, illegal escape sequences, malformed numeric literals, unexpected EOF, token overflow.
   [G]. Data Structure Description for tokenInfo (in maximum two lines): a struct containing token type, lexeme string, line & column numbers, plus an optional value field (union) for numeric or literal semantic value.
6. **Parser Details:**

[A]. High Level Data Structure Description (in maximum three lines each, avoid giving C definitions used):

i. grammar : The project grammar is stored as indexed productions where each RHS is an ordered array of symbol IDs (terminals/nonterminals).
   Productions include semantic action hooks used during parsing to build parse tree/AST.
   Nonterminals and terminals are mapped to compact integer IDs for fast table lookups.
   Found in grammar.txt

ii. FIRST and FOLLOW sets: FIRST and FOLLOW are computed iteratively from productions and stored as bitsets or hash-sets keyed by nonterminal ID.
   FIRST includes ε where applicable; FOLLOW is used for sync-based error recovery.

These sets drive LL(1) table construction and predictive parsing decisions.

iii. parse table: The parse table is a (nonterminal × terminal) → production-index mapping implemented as a sparse hash map (with vector-backed rows for dense parts).
Entries store the production index plus an optional action (sync/error) for recovery.
The table is generated from grammar + FIRST/FOLLOW and used for fast predictive lookups at parse time.

iv. parse tree: (Describe the node structure also): Parse-tree node: symbol ID, isTerminal flag, vector/list of child pointers, and (for terminals) pointer to tokenInfo (lexeme, line/col, value).
Nodes created on reductions preserve concrete derivation and include optional semantic attributes for later AST construction.
The tree supports traversal utilities for print, semantic actions and AST lowering.

v. Any other (specify and describe): AST: a simplified, typed tree derived from the parse tree with language-specific node kinds and attributes for semantic analysis and codegen.
Symbol table: scoped hash tables mapping identifiers to attribute records (type, declaration site, scope level) used in semantic checks.
Error recovery: FOLLOW-based sync plus token-position info from tokenInfo to report precise messages and continue parsing.

[B]. Parse tree

i. Constructed (yes/no): __yes__
ii. Printing as per the given format (yes/no): __yes__
iii. Describe the order you have adopted for printing the parse tree nodes (in maximum two lines)
In-order traversal (Left child, print node, remaining children)

[C]. Grammar and Computation of First and Follow Sets

i. Data structure for original grammar rules _indexed production table — each production = (LHS symbol ID, RHS vector of symbol IDs) stored in the grammar module for fast lookup.
ii. FIRST and FOLLOW sets computation automated (yes /no)_____yes_____
iii. Name the functions (if automated) for computation of First and Follow sets_____
compute_first_sets(), compute_follow_sets()_____
iv. If computed First and Follow sets manually and represented in file/function (name that) _____
_____N/A_____

[D]. Error Handling

v. Attempted (yes/ no):____yes___
Describe the types of errors handled: Syntax errors - unexpected tokens, missing required tokens (e.g., missing semicolon/closing bracket) with FOLLOW-based sync recovery
Lexical errors propagated from lexer - invalid token, unterminated string/comment
Recovery reports include line/column from tokenInfo for precise diagnostics

7. Compilation Details:

[A]. Makefile works (yes/no): ____yes_____
[B]. Code Compiles (yes/ no): _____yes_____
[C]. Mention the .c files that do not compile: _____none_____
[D]. Any specific function that does not compile: _____none_____
[E]. Ensured the compatibility of your code with the specified gcc version (yes/no) ____yes_____

8. Driver Details: Does it take care of the options specified earlier(yes/no): ___yes_____

9. Execution
   [A]. status (describe in maximum 2 lines):_____Runs successfully on the provided test harness; produces parse trees/ASTs, semantic checks and error reports and exits normally within expected runtime.
   [B]. Gives segmentation fault with any of the test cases (1-6) uploaded on the course page. If yes, specify the testcase file name: _____none__

10. Specify the language features your lexer or parser is not able to handle (in maximum one line)
    Our project currently cannot handle scope-related errors. (Missing variable declarations etc.) Basically we cannot take care of any context-specific tasks other than redeclaring the same variable as we have implemented a symbol table and check it.
11. Are you availing the lifeline (Yes/No): _____No_____

12. Declaration: We, Romit Jain, Soham Vinay Deshmukh, Nishant Amarish Pradhan, Devesh Saraogi (your names) declare that we have put our genuine efforts in creating the compiler project code and have submitted the code developed only by us. We have not copied any piece of code from any source. If our code is found plagiarized in any form or degree, we understand that a disciplinary action as per the institute rules will be taken against all of us in our team and we will accept the penalty as decided by the department of Computer Science and Information Systems, BITS, Pilani.

Your names and IDs

Name:____Romit Jain_____ID: _____2023A7PS0021P_____
Name:____Soham Vinay Deshmukh_____ ID: _____2023A7PS0025P_____
Name:____Nishant Amarish Pradhan___ ID: _____2023A7PS0030P_____
Name:____Devesh Saraogi_____ ID: _____2023A7PS0242P_____
Date: ____2nd March 2026_____

Not to exceed 3 pages.